

Logos Language Model Efficiency Lab

Experiment 2 Report: GELU Activation

Controlled activation-function comparison against the decoder-only GPT baseline

Run: logos_exp02_gelu_activation

Executive Summary

This report documents the second alternate method tested for the Logos language-model efficiency project: replacing the baseline ReLU activation in the feed-forward network with GELU. The goal was to test whether a smoother Transformer-style activation could improve validation loss and perplexity without changing model size. The experiment was controlled: the dataset, tokenizer, architecture size, optimizer, scheduler, random seed, and training budget were kept the same. Under this fixed 1,500-iteration budget, GELU did not improve the model. It produced higher validation loss and perplexity than the ReLU baseline while keeping parameter count and checkpoint size unchanged.

Result Area	Main Finding
Validation quality	Worse than baseline: final val loss 5.8091 vs 5.7634
Perplexity	Worse than baseline: 333.33 vs 318.42
Parameter efficiency	No improvement: parameter count stayed at 32.16M
Storage efficiency	No improvement: checkpoint size stayed at about 122.71 MB
Training speed	Slightly lower: 157,080 tok/s vs 158,641 tok/s
Generation latency	Worse in this run: 14.45 ms/token vs 10.20 ms/token
Decision	Reject GELU for the current fixed-budget Logos baseline; keep ReLU for now

1. Scientific Setup

The baseline model was a decoder-only GPT-style causal language model trained from scratch on a 10,000-sample OpenWebText subset using the GPT-2 tokenizer. Experiment 2 kept the same model size and training setup but changed the feed-forward activation from ReLU to GELU. This makes the experiment a one-variable controlled comparison.

Scientific Component	Description
Control condition	Original Logos decoder-only GPT baseline using ReLU
Experimental condition	Same model using GELU in the feed-forward network
Independent variable	Activation function: ReLU changed to GELU
Controlled variables	Dataset, tokenizer, seed, depth, heads, embedding dimension, batch size, iterations, optimizer, scheduler, dropout
Primary dependent metrics	Validation loss, perplexity, speed, VRAM, latency, checkpoint size, and energy
Hypothesis	GELU may improve validation loss without increasing parameter count

2. Method: ReLU to GELU

The baseline feed-forward network used ReLU, which passes positive values directly and sets negative values to zero. GELU is a smoother nonlinear activation commonly used in Transformer models. It can sometimes improve optimization and representation quality, but it may also be slightly more expensive computationally.

```
Baseline FFN:  
Linear -> ReLU -> Linear -> Dropout
```

```
GELU experiment FFN:  
Linear -> GELU -> Linear -> Dropout
```

No weight tying was used in this experiment. This was intentional, because the purpose was to compare GELU directly against the original ReLU baseline with the same parameter count.

```
Changed only:  
nn.ReLU() -> nn.GELU()
```

```
Weight tying active: False
```

3. Architecture and Training Configuration

Configuration Item	Value
Model type	Decoder-only GPT-style causal language model
Experiment type	GELU activation
Tokenizer	GPT-2 tokenizer
Dataset	OpenWebText subset
Dataset samples	10,000 text samples
Vocabulary size	50,257
Total tokens	11,153,991
Train tokens	10,038,591
Validation tokens	1,115,400
Layers	8
Attention heads	8
Embedding dimension	256
Context length	256 tokens
Dropout	0.2
Activation	GELU
Weight tying	False
Batch size	64
Max iterations	1,500
Optimizer	AdamW
Learning rate	3e-4
Weight decay	0.1
Gradient clipping	1.0
Mixed precision	Enabled

4. Hardware and Software Environment

Item	Value
Device	CUDA GPU
GPU	NVIDIA RTX 6000 Ada Generation
Available GPU memory	47.99 GB
CUDA version	12.1
PyTorch version	2.5.1+cu121

5. GELU Run Results

Metric	Value
Run name	logos_exp02_gelu_activation
Parameters	32,159,825
Trainable parameters	32,159,825
Activation	GELU
Weight tying active	False
Best iteration	1,500
Best validation loss	5.8166
Final training loss	5.6279
Final validation loss	5.8091
Final validation perplexity	333.33
Checkpoint size	122.71 MB
Peak VRAM	11095 MB, about 11.10 GB
Training time	157.25 seconds, about 2.62 minutes
Training speed	157,080 tokens/sec
Generation speed	69.20 tokens/sec
Generation latency	14.45 ms/token
Average GPU power	251.57 W
Estimated GPU energy	10.84 Wh

6. Training Behavior

The GELU model continued improving until the final iteration, and the best checkpoint was saved at iteration 1,500. However, its validation loss remained higher than the original ReLU baseline at the same training budget. This means the model was still learning, but it was not better than the control run within the same fixed compute budget.

Iteration	Training Loss	Validation Loss	Validation Perplexity
0	10.8651	10.8662	52,374.26
150	6.9227	6.9716	1,065.91
300	6.3831	6.4640	641.64

450	6.1461	6.2506	518.35
600	5.9863	6.0992	445.48
750	5.8833	6.0305	415.92
900	5.7774	5.9200	372.39
1050	5.7123	5.8854	359.75
1200	5.6564	5.8330	341.40
1350	5.6452	5.8294	340.16
1500	5.6308	5.8166	335.84

7. Comparison Against Baseline

Metric	Baseline ReLU	GELU	Change
Parameters	32.16M	32.16M	No change
Final validation loss	5.7634	5.8091	Worse by +0.0457
Final validation perplexity	318.42	333.33	Worse by +14.91
Checkpoint size	122.71 MB	122.71 MB	No change
Peak VRAM	10.84 GB	11.10 GB	Higher
Training time	156.42 sec	157.25 sec	Slightly slower
Training speed	158,641 tok/s	157,080 tok/s	-0.98% lower
Generation latency	10.20 ms/token	14.45 ms/token	41.67% slower
Energy	10.78 Wh	10.84 Wh	0.56% higher

8. Interpretation

GELU did not provide an efficiency improvement in this fixed-budget run. It preserved the same parameter count and checkpoint size, but it had worse validation loss, worse perplexity, slightly higher memory usage, slower generation, and similar energy usage. Therefore, under the current Logos baseline setup, ReLU is the better activation choice.

This does not mean GELU is generally worse for all language models. GELU often works well in larger Transformer models, and it may perform differently with longer training, different learning rates, or altered initialization. However, the purpose of this experiment was to make a controlled decision under the same budget, and under that condition GELU should not replace ReLU in the current Logos stack.

9. Scientific Conclusion

Experiment 2 rejects the hypothesis that simply replacing ReLU with GELU improves the current Logos baseline under the same training budget. The GELU model achieved a final validation loss of 5.8091 and perplexity of 333.33, while the ReLU baseline achieved 5.7634 and 318.42. Because the parameter count and checkpoint size did not change, the worse validation quality means GELU reduced the capability-resource tradeoff in this particular run.

Decision	Recommendation
Keep or reject?	Reject GELU for current fixed-budget Logos baseline

Reason	Higher validation loss and perplexity with no parameter or storage reduction
Caution	May be retested later with longer training or tuned hyperparameters
Next experiment	RMSNorm instead of LayerNorm

10. Next Experiment

The next clean experiment should replace LayerNorm with RMSNorm while keeping the rest of the model unchanged. This tests a normalization method often used in efficient decoder-only language models. The expected benefit is similar validation quality with slightly simpler normalization and possibly improved efficiency.

Experiment 3: RMSNorm

Change only:

```
nn.LayerNorm(N_EMBED) -> RMSNorm(N_EMBED)
```

Keep fixed:

dataset, tokenizer, seed, batch size, context length, model depth, embedding size, heads, activation, optimizer, scheduler, and max iterations

One-Line Summary

GELU did not improve the current Logos baseline: it kept the same 32.16M parameters and 122.71 MB checkpoint size, but worsened final validation loss from 5.7634 to 5.8091 and perplexity from 318.42 to 333.33 under the same fixed training budget.